

MORE-STEM Figure 2-5

超详细模块学习文档

逐模块输入/输出/维度/含义/模型/训练状态/损失函数设计

适用范围：本文档只解释 MORE-STEM 论文中 Figure 2、Figure 3、Figure 4、Figure 5 及 Methodology 3.1-3.3 中明确出现的模块，并把论文未给出但复现时必须关心的维度与实现细节用符号化方式补齐。

阅读约定：论文主文没有给出所有数值型超参数，例如统一隐藏维度 D、image encoder backbone、memory bank 容量 M、short-term window K、classifier loss 的完整数学式。本文档凡是论文未明示的地方，均标注为“主文未明确”。维度统一采用符号形式，不能当作论文给定的固定数值。

0. 先给结论：Figure 2-5 的职责分工

图	核心机制	解决的问题
Figure 2 总框架	把点云、图像、文本三种输入编码成四类特征；依次经过 CFTVA、STEM、Transformer Enc/Dec、Long-Short Memory、Classifier。	回答“整体怎么从输入到 mask”。
Figure 3 CFTVA	跨帧文本-视觉对齐。把文本 query 与当前帧/历史帧的点云和图像特征对齐。	回答“query 说的是哪个目标”。
Figure 4 STEM	时空一致性建模。用 state-space 做帧间传播，用 Transformer 做帧内空间精修。	回答“目标在连续帧中如何不抖、不漂、不跳”。
Figure 5 Long-Short Memory	长期记忆召回历史文本-视觉对；短期记忆缓存最近 mask 特征。	回答“以前见过什么”和“上一帧在哪里”。

1. 统一符号、维度和数据结构

先统一符号，否则图 2-5 很容易混淆。下表中的维度是为解释和复现而定义的通用符号，不是论文报告的固定数值。

符号	维度/类型	含义
B	batch size	训练/推理时一次输入的序列数量。论文图中省略 batch 维。
T	sequence length	输入帧数。3D 任务可取 $T=1$ ，或按论文实验方式把同一场景重复输入多次；4D 任务 $T>1$ 。
t	frame index	当前帧索引。图 3 使用 t、t-1、t-2；图 4 使用 t 和 t-1。
P_t	$R^{N_t \times C_p}$	第 t 帧点云。N_t 是点数，C_p 是原始点属性维度，通常至少包含 x,y,z，可包含 intensity、timestamp 等。
I_t	$R^{H \times W \times 3}$	第 t 帧 RGB 图像，与 LiDAR 点云时间/外参对齐。
q	token ids length L	文本 query，例如“identify the car moving opposite...”。
N_t	integer	第 t 帧点数，通常每帧不同。
V_t	integer	体素化后的稀疏 voxel token 数。
M_t	integer	图像 encoder 输出的图像 token 数，例如 patch token 数。
L	integer	文本 token 数。
D_p	feature dim	点级特征通道数。

符号	维度/类型	含义
D_v	feature dim	体素级特征通道数。
D_i	feature dim	图像特征通道数。
D_llm	feature dim	LLM 输出的文本隐藏维度。若按常见 LLaMA2-7B 设定为 4096，但论文正文未说明具体取法。
D	feature dim	CFTVA 投影后的统一 latent dimension。文本/视觉/点云都要投影到该空间才能 attention。
D_s	feature dim	STEM state-space hidden state 维度；可与 D 相同，也可不同。
M	memory size	Long-Term Memory Bank 中保留的历史条目数。
K	short window	Short-Term Mask Memory Bank 缓存的最近 mask feature 数。
C_cls	class count	普通 query-driven 二值分割时 C_cls=1；多类别语义分割变体时为类别数。

1.1 输出 mask 的形状

- 点级二值 mask: $\text{logits}_t \in \mathbb{R}^{N_t \times 1}$ ，sigmoid 后得到每个点属于 query 目标的概率。
- 体素级二值 mask: $\text{logits}_t \in \mathbb{R}^{V_t \times 1}$ ，若评价需要点级结果，则要把 voxel logits scatter/interpolate 回点。
- 4D mask: $\{\text{logits}_t\}_{t=1}^T$ ，可写为 $\mathbb{R}^{T \times N_t \times 1}$ ；因为 N_t 可变，实际实现常用稀疏 batch 或列表。
- 多类别语义分割变体: $\text{logits}_t \in \mathbb{R}^{N_t \times C_{\text{cls}}}$ 或 $\mathbb{R}^{V_t \times C_{\text{cls}}}$ 。

2. Figure 2: 总体网络框架逐模块解释

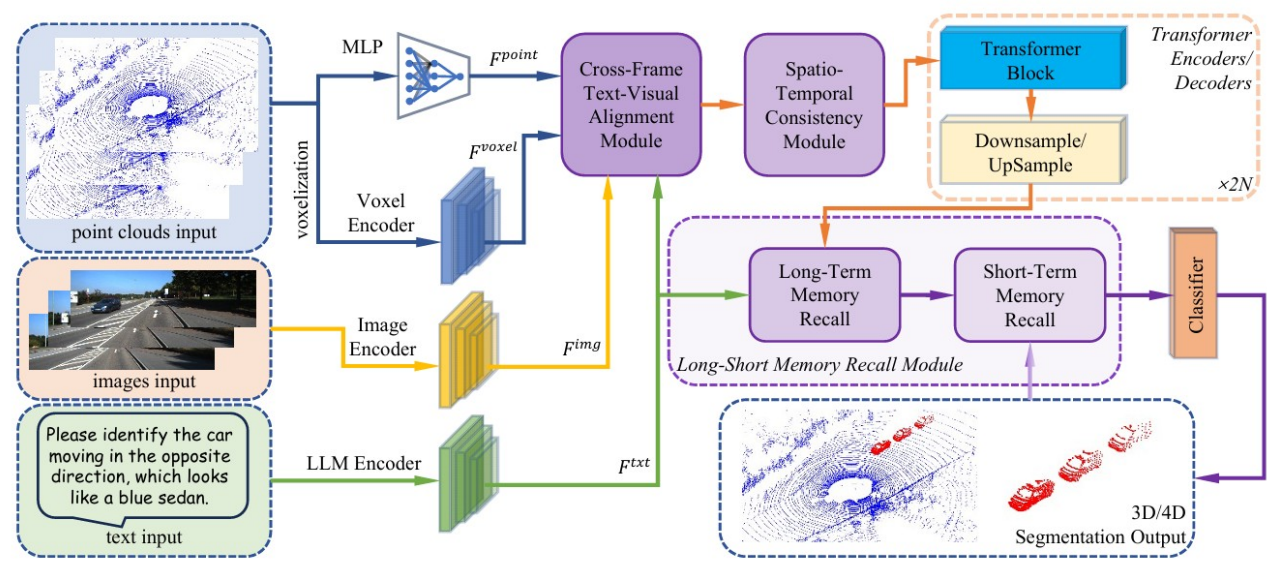


Figure 2. A general overview of the proposed network. Given multi-frame point clouds, RGB images, and a text query, four feature types (f_{point} , f_{voxel} , f_{img} , f_{txt}) are extracted and aligned through the Cross-Frame Text-Visual Alignment (CFTVA) module. The Spatio-Temporal Consistency Model (STEM) module enforces motion-aware temporal coherence using a state-space model and sparse 3D Transformer. The Long-Short Memory Recall mechanism retrieves both long-term text-visual pairs and short-term mask features to enhance temporal reasoning. Finally, a segmentation head outputs query-driven 3D/4D masks with spatial and temporal consistency.

Figure 2: MORE-STEM 总体网络。左侧输入点云、图像、文本；中间完成特征提取、跨模态对齐、时空一致性和记忆召回；右侧输出 query-driven 3D/4D mask。

2.1 Figure 2 总体数据流

Input: $\{P_t\}_{t=1..T}$, $\{I_t\}_{t=1..T}$, text query q

Point branch: $P_t \rightarrow \text{MLP} / \text{point encoder} \rightarrow F_t^{\text{point}}$
Voxel branch: $P_t \rightarrow \text{voxelization} \rightarrow \text{voxel encoder} \rightarrow F_t^{\text{voxel}}$
Image branch: $I_t \rightarrow \text{image encoder} \rightarrow F_t^{\text{img}}$
Text branch: $q \rightarrow \text{frozen LLaMA2-7B} \rightarrow F_t^{\text{txt}}$

CFTVA: $(F_t^{\text{point}}, F_t^{\text{voxel}}, F_t^{\text{img}}, F_t^{\text{txt}}, \text{history frames}) \rightarrow \text{aligned features}$
STEM: $\text{aligned features} + \text{previous hidden state} \rightarrow \text{spatio-temporal feature } \tilde{f}_t$
Transformer Enc/Dec: $\tilde{f}_t \rightarrow \text{multi-scale segmentation feature}$
Long-Short Memory: $\text{feature} + \text{memory banks} \rightarrow \text{memory-enhanced feature}$
Classifier: $\text{memory-enhanced feature} \rightarrow \text{query-driven 3D/4D segmentation mask}$

2.2 输入分支：每类输入到底提供什么信息

输入	形状	提供的信息	去向
Point Clouds Input	$P_t \in \mathbb{R}^{N_t \times C_p}$	3D 几何、距离、稀疏结构、目标在连续帧的位置变化。	用于点级和体素级特征提取。
Images Input	$I_t \in \mathbb{R}^{H \times W \times 3}$	颜色、纹理、外观属性。例如 blue sedan、red shirt、silver car。	用于图像编码，并在 CFTVA 中增强 3D 特征。
Text Input	$q, \text{length } L$	目标类别、属性、空间关系、运动关系。例如 car + opposite direction + blue sedan。	由 frozen LLaMA2-7B 编码成 F^{txt} 。

2.3 点云点级分支：MLP / Point Encoder

项目	内容	解释
输入	$P_t \in \mathbb{R}^{N_t \times C_p}$	每个点的原始几何/传感器属性。
输出	$F_t^{\text{point}} \in \mathbb{R}^{N_t \times D_p}$	每个点一个 embedding，保留点级边界、局部

项目	内容	解释
		形状、细粒度几何。
模型	图中写 MLP；正文说 per-point embeddings 来自 lightweight Point Transformer。	可理解为：MLP 做初始特征升维，点级 Transformer/point branch 建模点间关系。
训练状态	训练	论文未说冻结，属于主网络可学习分支。
为什么需要	体素化会损失部分细节；点级分支保留目标边界和小物体细节。	最终 mask 通常需要精确到点。

2.4 点云体素分支：Voxelization + Voxel Encoder

项目	内容	解释
Voxelization 输入	$P_t \in R^{N_t \times C_p}$	原始点云。
Voxelization 输出	稀疏 voxel tokens: $X_t^{voxel} \in R^{V_t \times C_v}$	V_t 是非空 voxel 数； C_v 可由 voxel 内点聚合得到。
Voxelization 是否训练	不训练	它是几何离散化/索引操作，不是神经网络层。
Voxel Encoder 输入	$X_t^{voxel} \in R^{V_t \times C_v}$	稀疏体素特征。
Voxel Encoder 输出	$F_t^{voxel} \in R^{V_t \times D_v}$	体素级上下文特征。
模型	sparse 3D Transformer with window shifting attention	论文正文明确提到稀疏 3D Transformer 和窗口移动注意力。
训练状态	训练	没有声明冻结；属于点云主干。

2.5 图像分支：Image Encoder

项目	内容	解释
输入	$I_t \in R^{H \times W \times 3}$	与点云对齐的 RGB 图像。
输出	$F_t^{img} \in R^{M_t \times D_i}$	图像 token 特征， M_t 取决于 image encoder 的 patch/feature map 设计。
作用	补充外观语义	点云难以识别颜色和纹理，图像能提供 blue/silver/red 等属性。
模型	主文只写 Image Encoder	没有明确是 ResNet、ViT、CLIP image encoder 或其他 backbone。
训练状态	主文未明确	不能断言冻结。实现中可能端到端训练、fine-tune 或冻结预训练图像 encoder，需要看代码/补充材料。

2.6 文本分支：LLM Encoder

项目	内容	解释
输入	text query q, token length L	自然语言指令。
输出	$F^{txt} \in R^{L \times D_{llm}}$ 或 pooled $R^{D_{llm}}$	文本 token 或句向量。主文没有说明取最后 token、mean pooling 还是专门 pooling。
模型	frozen, pretrained LLaMA2-7B	论文明确说明文本 query 使用冻结的预训练 LLaMA2-7B 编码。
训练状态	LLM 本体冻结	后续线性投影 W_{txt} 训练。
作用	解析 query 条件	包括类别 car、外观 blue sedan、运动 moving opposite、空间 closer 等。

2.7 Figure 2 中各模块的训练/冻结状态总览

模块	状态	依据/说明
MLP / Point branch	训练	图中和正文都属于主网络编码器。
Voxelization	不训练	几何离散化操作。
Voxel Encoder	训练	稀疏 3D Transformer 主干。
Image Encoder	主文未明确	只能说输出 F_{img} ；冻结/微调需看补充材料或代码。
LLaMA2-7B	冻结	主文明确 frozen, pretrained LLaMA2-7B。
Linear projection W_{txt}, W_{vis}	训练	把不同模态映射到统一维度 D。

模块	状态	依据/说明
CFTVA attention/fusion	训练	包括 cross-attention、cross-frame attention、contrastive alignment。
STEM state-space/Transformer	训练	A、B、Transformer、Norm/fusion 可学习。
Transformer Enc/Dec	训练	分割网络编码解码层。
Memory banks	动态维护，不是普通可学习参数	存储历史特征或最近 mask feature。
Memory attention/fusion	训练	Self-attention、cross-attention、classifier 前融合层可学习。
Classifier	训练	输出 logits 并计算分割损失。

3. Figure 3: CFTVA 跨帧文本-视觉对齐模块

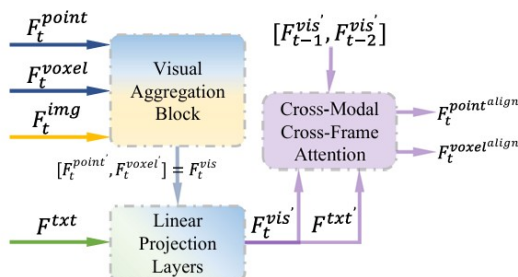


Figure 3. Framework of the proposed Cross-Frame Text-Visual Alignment module.

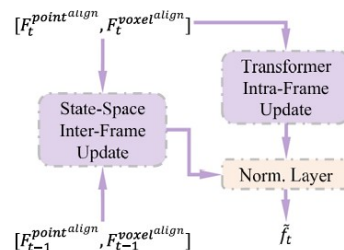


Figure 4. Framework of the proposed Spatio-Temporal Consistency Model module.

Figure 3 位于左侧: CFTVA。它把 F_point 、 F_voxel 、 F_img 、 F_txt 对齐, 并输出 $F_t^{point-align}$ 和 $F_t^{voxel-align}$ 。

3.1 CFTVA 的目标

CFTVA 不是普通的“文本-图像对齐”。它同时做三件事:

1. 2D-3D 外观/几何对齐: 让点云特征吸收图像颜色和外观语义。
2. 文本-视觉对齐: 让 query 语义选择相关的点云/图像 token。
3. 跨帧对齐: 让文本中的 motion words, 例如 moving opposite, 通过 t 、 $t-1$ 、 $t-2$ 的视觉 token 得到解释。

3.2 CFTVA 输入输出

变量	维度	含义
F_t^{point}	$R^{\{N_t \times D_p\}}$	点级 token。每个点一个特征, 保留局部几何。
F_t^{voxel}	$R^{\{V_t \times D_v\}}$	体素 token。每个非空 voxel 一个特征, 保留 3D 上下文。
F_t^{img}	$R^{\{M_t \times D_i\}}$	图像 token。每个 patch/feature-map 位置一个视觉特征。
F^{txt}	$R^{\{L \times D_llm\}}$ 或 $R^{\{D_llm\}}$	文本 token 或句向量, 由 frozen LLaMA2-7B 给出。
$F_t^{point-align}$	$R^{\{N_t \times D\}}$ 或投影后的点级 aligned feature	query-aware 点级特征: 不仅知道几何, 还知道与文本目标的相关性。
$F_t^{voxel-align}$	$R^{\{V_t \times D\}}$	query-aware 体素级特征: 后续 STEM 主要在 voxel-level 做时序传播。

3.3 步骤 1: Visual Aggregation Block

论文公式写为:

$$\begin{aligned}
 F_t^{point'} &= \text{CrossAttn}(F_t^{point}, F_t^{img}) \\
 F_t^{voxel'} &= \text{CrossAttn}(F_t^{voxel}, F_t^{img}) \\
 F_t^{vis} &= [F_t^{point'}, F_t^{voxel'}]
 \end{aligned}$$

更具体地说, $\text{CrossAttn}(A, B)$ 可以理解为 A 作为 query, B 作为 key/value:

$$\begin{aligned}
 Q &= A W_Q, \quad K = B W_K, \quad V = B W_V \\
 \text{Attn}(A, B) &= \text{softmax}(Q K^T / \sqrt{D}) V
 \end{aligned}$$

子步骤	Q/K/V	输出	含义
点级对图像注意力	Q: $F_t^{point} \in R^{\{N_t \times D_p\}}$; K,V: $F_t^{img} \in R^{\{M_t \times D_i\}}$	输出 $F_t^{point'} \in R^{\{N_t \times D\}}$	每个点从图像 token 中取与自己相关的颜色/外观语义。
体素级对图像注意力	Q: $F_t^{voxel} \in R^{\{V_t \times D_v\}}$; K,V: $F_t^{img} \in R^{\{M_t \times D_i\}}$	输出 $F_t^{voxel'} \in R^{\{V_t \times D\}}$	每个体素从图像 token 中取与自己相关的视觉上下文。

注意：论文文字说 CrossAttn 表示 3D 与 2D 模态之间的 bidirectional attention，但公式只展开了点/体素特征由图像特征增强的一侧。若复现，需要查看代码确认是否还有 image-to-3D 的反向更新。

3.4 步骤 2: Linear Projection Layers

不同模态的特征维度不同，不能直接做 attention。投影层把文本和视觉映射到同一个维度 D：

$$F^{txt'} = W_{txt} F^{txt}$$
$$F_{t'}^{vis} = W_{vis} F_t^{vis}$$

投影层	参数维度	输出	训练状态
W_{txt}	$R^{D_{llm} \times D}$ 或对 token 逐位置线性投影	$F^{txt'} \in R^{L \times D}$ 或 R^D	训练
W_{vis}	$R^{D_{vis} \times D}$	$F_{t'}^{vis} \in R^{(N_t+V_t) \times D}$ 或分流保持 $R^{N_t \times D}, R^{V_t \times D}$	训练

关键实现歧义： $F_{t'}^{vis} = [F_{t'}^{point}, F_{t'}^{voxel}]$ 的中括号可以表示拼接，也可以表示保留两条分支。由于 point token 数 N_t 和 voxel token 数 V_t 不同，真实实现中通常需要沿 token 维拼接、或分别投影后在后续模块中维护映射关系。

3.5 步骤 3: Cross-Modal Cross-Frame Attention

论文公式写为：

$$F_{t'}^{vis-align} = \text{Attn}(F^{txt'}, [F_{t'}^{vis}, F_{t-1}^{vis}, F_{t-2}^{vis}])$$
$$F_{t'}^{vis-align} = [F_{t'}^{point-align}, F_{t'}^{voxel-align}]$$

角色	变量	含义
Query	$F^{txt'}$	文本 query。若是 token 级，则形状 $R^{L \times D}$ ；若 pooled，则 $R^{1 \times D}$ 。
Key/Value	$[F_{t'}^{vis}, F_{t-1}^{vis}, F_{t-2}^{vis}]$	当前帧和前两帧视觉/3D token，形状约 $R^{S \times D}$ ， $S=(N_t+V_t)+(N_{t-1}+V_{t-1})+(N_{t-2}+V_{t-2})$ 。
Output	$F_{t'}^{vis-align}$	被文本 query 选择和调制后的视觉/点云特征，再拆成 point-align 和 voxel-align。

为什么需要 t-1、t-2？因为单帧无法判断“moving opposite”。只有比较连续帧中目标位置变化，模型才能把运动词和具体对象绑定。

3.6 CFTVA 的损失函数: Temporal Contrastive Loss

论文明确给出了 CFTVA 的对齐损失：

$$L_{align} = - \log [\exp(\text{sim}(F_{t'}^{vis-align}, F^{txt'})) / \tau] / \sum_{t'} \exp(\text{sim}(F_{t'}^{vis-align}, F^{txt'})) / \tau]$$

符号/概念	含义	解释
$\text{sim}(.,.)$	cosine similarity	度量文本和视觉对齐特征的相似度。
τ	temperature	温度系数，控制 softmax 分布尖锐程度。
positive	$F_{t'}^{vis-align}$ 与当前 query	正确帧/正确目标的视觉特征应该接近文本。
negative	$F_{t'}^{vis-align}, t' \neq t$ 或其他不匹配特征	不相关帧/不相关目标应该远离文本。
作用	强化判别性对齐	让文本 query 更稳定地锁定动态目标，而不是只靠 classifier 学。

3.7 CFTVA 输出的语义变化

- 输入 $F_{t'}^{point}$ ：只是点级几何特征，知道“这里像一辆车的一部分”。

- 输入 F_t^{voxel} : 只是体素级结构特征, 知道 “这里是一个车辆形状区域”。
- 输入 F_t^{img} : 知道 “这里可能是蓝色车/红衣行人”。
- 输入 F_t^{txt} : 知道 query 要找 “反方向移动的蓝色轿车”。
- 输出 $F_t^{\{\text{point-align}\}}, F_t^{\{\text{voxel-align}\}}$: 变成 query-aware 特征, 知道 “这些点/体素更符合 query 描述的目标”。

4. Figure 4: STEM 时空一致性模块

STEM 全称是 Spatio-Temporal Consistency Model。它接收 CFTVA 输出的 aligned features，进一步保证连续帧之间目标一致、当前帧内部空间结构合理。

4.1 STEM 输入输出

类型	变量/维度	含义
输入 1	$F_t^{\text{point-align}} \in R^{N_t \times D}$	当前帧 query-aware 点级特征。
输入 2	$F_t^{\text{voxel-align}} \in R^{V_t \times D}$	当前帧 query-aware 体素级特征。
历史状态	$h_{t-1}(v) \in R^{D_s}$	上一帧传播来的 voxel-level hidden state。图中用 t-1 的 aligned features 表达历史信息。
输出	$f_{\text{tilde}_t}(v) \in R^{D_s}$	时空一致性增强后的 voxel 特征，供后续编码解码和分割使用。

4.2 点级特征如何进入 voxel-level state

主文未明确：公式 $h_t(v)$ 同时使用 $F_t^{\text{point-align}}(v)$ 和 $F_t^{\text{voxel-align}}(v)$ 。但是 point token 和 voxel token 的索引不同。真正实现需要把点级特征聚合到 voxel v ，常见做法是 voxel 内点池化、scatter mean/max、最近邻映射或维护 point-to-voxel index。

```
For each voxel v:
    point features inside voxel v: {F_t^{point-align}(p) | p belongs to voxel v}
    pooled point feature: P_t(v) = mean/max/attention_pool(...)
    voxel feature:          V_t(v) = F_t^{voxel-align}(v)
    x_t(v) = concat(P_t(v), V_t(v)) ∈ R^{2D}
```

4.3 State-Space Inter-Frame Update：帧间状态传播

论文公式为：

$$h_t(v) = A \ h_{t-1}(v) + B \ [F_t^{\text{point-align}}(v) \ || \ F_t^{\text{voxel-align}}(v)]$$

符号	维度	含义
v	voxel index	当前帧中的某个体素位置。
$h_{t-1}(v)$	R^{D_s}	上一帧的时间隐藏状态，携带历史信息。
$x_t(v)=[\text{point} \text{voxel}]$	R^{2D}	当前帧观测到的 query-aware 点级+体素级特征。
A	$R^{D_s \times D_s}$	状态转移矩阵，控制历史状态保留多少。训练。
B	$R^{D_s \times 2D}$	输入映射矩阵，控制当前观测如何写入状态。训练。
$h_t(v)$	R^{D_s}	更新后的时间状态。

直观解释：当前帧特征不是独立预测，而是“上一帧记忆 + 当前帧观测”。这能减少 mask 抖动和目标跳变。

4.4 Transformer Intra-Frame Update：帧内空间精修

论文公式为：

$$z_t(v) = \text{Transformer}(Q_t, K_t, V_t)$$

项目	内容	解释
Q_t, K_t, V_t	通常由 h_t 或 aligned feature 线性投影得到	用于当前帧内部 attention。
$z_t(v)$	R^{D_s}	当前 voxel 聚合邻近 voxel / 相关 voxel 后的空间上下文特征。
作用	帧内空间一致性	保证同一目标区域连续，减少车头分出来车尾漏掉、边界粘连等问题。
模型	voxel-level / sparse 3D Transformer	论文说使用 controllable 3D Transformer for

项目	内容	解释
		sparse intra-frame attention。
训练状态	训练	Transformer 权重可学习。

4.5 Norm Layer：融合时间和空间信息

论文公式为：

$$f_tilde_t(v) = \text{Norm}(h_t(v) + z_t(v))$$

- $h_t(v)$ ：跨帧状态空间传播结果，强调时间连续性。
- $z_t(v)$ ：当前帧 Transformer 精修结果，强调空间结构一致性。
- Norm：稳定训练，融合两路信息，输出后续分割需要的 refined spatio-temporal feature。

4.6 STEM 是否有单独损失

论文主文没有给 STEM 单独定义一个 L_STEM 。STEM 的参数主要通过后续 segmentation/classifier loss 以及整体任务监督反向传播学习。它的作用由消融实验验证：去掉 STEM 后性能下降，说明时空一致性建模有效。

4.7 STEM 的实现注意点

- 如果物体运动较大，voxel v 在 $t-1$ 和 t 帧中可能不对应同一物理点。论文主文没有给出显式 scene flow 或 motion compensation，因此状态传播是否基于坐标对齐、ego-motion 对齐或稀疏索引，需要查看代码。
- A 、 B 可以是全局矩阵，也可以是通道门控/逐层参数。主文只写 learnable transition parameters，没有展开。
- Transformer intra-frame update 如果用全局 attention，复杂度高；实际更可能是 sparse/local attention。主文使用 sparse intra-frame attention 的描述。

5. Figure 5: Long-Short Memory Recall 长短期记忆召回模块

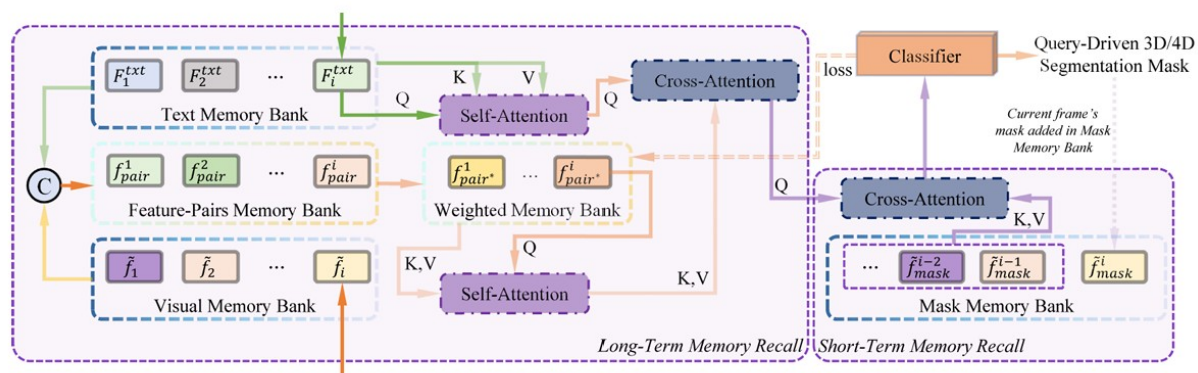


Figure 5. Framework of the proposed Long-Short Memory Recall module.

Figure 5: 长短期记忆召回模块。左侧 LTM 存历史文本/视觉/文本-视觉对；右侧 STM 存最近 mask feature；最后由 classifier 输出 segmentation mask。

5.1 这个模块解决什么

- Long-Term Memory Recall：跨场景语义召回。目标是“当前 query 与历史中哪些文本-视觉对应关系相似”。
- Short-Term Memory Recall：当前序列短期连续性。目标是“最近几帧这个目标的 mask 在哪里、形状如何”。
- Classifier：融合 LTM 和 STM 后，输出当前 query 的 3D/4D mask。

5.2 长期记忆库的三类 bank

Memory bank	符号	维度/形式	存储含义
Text Memory Bank	$\{F_i^{txt}\}_{i=1..M}$	每条通常可存 pooled 文本向量 R^D ，也可存 token 表示。	历史 query 的语义。
Visual Memory Bank	$\{\tilde{f}_i\}_{i=1..M}$	每条通常存目标/场景级视觉向量 R^D 。	历史目标的视觉/3D/时空特征。
Feature-Pairs Memory Bank	$\{f_{pair}^i\}_{i=1..M}$	$f_{pair}^i = [F_i^{txt}, \tilde{f}_i]$ ，可为 R^{2D} 或再投影为 R^D 。	历史“文本描述 -> 视觉目标”的绑定关系。
Weighted Memory Bank	$\{f_{pair}^{i*}\}$	$f_{pair}^{i*} = w_i * f_{pair}^i$	按置信度和类别平衡调整后的可召回记忆。

5.3 长期记忆条目的权重设计

论文给出两个权重公式。第一个是按样本 loss 的置信度权重：

$$w_i^{init} = 1 / (L_i + \epsilon)$$
$$M_{LTM} = \{ w_i * (f_{pair}^i : [F_i^{txt}, \tilde{f}_i]) \}$$

含义：L_i 越小，说明该历史样本越可靠，权重越大。

第二个是类别平衡权重：

$$w_i^{bias} = w_i^{init} / \sum_{j \in c} w_j^{init}$$

- c 表示语义类别。若 car 样本很多，直接存储会导致 car 记忆支配召回。
- 类别内归一化使同一类别的总权重受控，缓解频繁类别过度表示。
- 论文主文说同时维护 w_i^{init} 和 w_i^{bias} ，但没有展开最终 w_i 如何组合；可理解为 weighted memory 使用置信度和类别平衡后的权重。

5.4 LTM 推理流程：Self-Attention + Cross-Attention

根据论文正文，LTM 的推理过程可以拆成三步。

流程	Q/K/V	输出	直观含义
步骤 1: 文本记忆检索	Q: 当前 F^{txt} ; K,V: Text Memory Bank	Self-Attention 输出文本语义上下文 c_{txt} 。	找到历史中语义相似的 query。
步骤 2: 文本-视觉对记忆检索	Q: 当前场景/加权特征; K,V: Feature-Pairs / Visual / Weighted Memory	Self-Attention 输出多模态记忆上下文 c_{pair} 。	找到历史中相似 query 对应的视觉目标经验。
步骤 3: 长期记忆融合	Q: 第一步输出; K,V: 第二步输出	Cross-Attention 输出 r_{LTM} 。	把文本召回和视觉召回结合, 得到长期语义增强特征。

```
c_txt = SelfAttn(Q=F_txt_current, K=TextMemory, V=TextMemory)
c_pair = SelfAttn(Q=current_or_weighted_feature, K=PairMemory, V=VisualMemory/PairMemory)
r_LTM = CrossAttn(Q=c_txt, K=c_pair, V=c_pair)
```

主文细节限制：图 5 对 Q/K/V 画得比较抽象，正文也没有给出精确张量实现。上面的伪代码是依据图示和文字整理出的可复现理解，而不是论文逐行代码。

5.5 短期记忆 STM: Mask Memory Bank

项目	符号/变量	含义
Mask Memory Bank	$\{\tilde{mask}^{i-k}\}_{k=1..K}$	最近 K 帧的 mask feature。每条可为 R^D 的目标级向量，也可为 $R^{N_{mask} \times D}/R^{V_{mask} \times D}$ 的区域特征。
当前输入 Q	r_{LTM}	长期记忆增强后的当前特征。
K,V	Mask Memory Bank	最近帧 mask 特征，包含目标形状、位置、短期运动趋势。
输出	r_{STM}	短期时序增强特征，送入 classifier。

STM 的 cross-attention 可写成：

```
r_STM = CrossAttn(Q = r_LTM, K = MaskMemory, V = MaskMemory)
```

预测当前帧后，当前帧的 mask feature 会写入 Mask Memory Bank：

```
MaskMemory <- update(MaskMemory, f_tilde_mask^i)
# 通常保留最近 K，最的移出。
```

5.6 Classifier 和 classifier loss

项目	变量/形式	解释
输入	r_{STM} 或 memory-enhanced feature	融合长期语义和短期 mask 记忆后的特征。
输出	$logits \in R^{N_t \times 1}$ 或 $R^{V_t \times 1}$	每个点/体素属于 query 目标的概率 logits。
监督	ground-truth 3D/4D mask	由数据集标注提供。
loss	主文没有给出完整公式	Figure 5 标了 classifier loss，但正文未展开是 CE、BCE、Dice、Lovasz 还是组合。

不要误读：不能把 $w_i^{init}=1/(L_i+\epsilon)$ 当成最终分割训练损失。它是长期记忆条目的权重计算方式。真正的分割监督来自 classifier 输出和 ground-truth mask 的比较，但主文没有给出具体公式。

5.7 一个完整例子：找反方向移动的蓝色车

- 4. 文本编码器得到 F_{txt} ，包含 car、blue sedan、moving opposite 等语义。
- 5. CFTVA 已经把 query 与当前帧和历史帧视觉/点云 token 对齐。
- 6. STEM 已经把连续帧的 voxel 特征做了时空一致性更新。
- 7. 进入 LTM 后，Text Memory Bank 召回历史中类似“blue sedan / moving opposite”的 query。
- 8. Feature-Pairs Memory Bank 进一步召回这些 query 当时对应的视觉/3D 目标特征。
- 9. 进入 STM 后，Mask Memory Bank 提供最近几帧该目标 mask 的位置和形状。

10. Classifier 综合长期语义经验与短期 mask 连续性，输出当前帧或多帧的目标 mask。

5.8 LTM 与 STM 的差异：为什么二者都需要

对比项	Long-Term Memory Recall	Short-Term Memory Recall
记忆范围	跨样本、跨场景、长周期	当前序列内、最近 K 帧
记忆内容	文本特征、视觉/3D 特征、文本-视觉配对	最近帧的 mask feature
主要解决	语义泛化和历史经验召回	目标连续性和短期跟踪稳定
典型错误修正	多个外观相似目标中，借助历史文本-视觉对应关系增强识别	上一帧已经选中 A 车，当前帧避免跳到 B 车
更新方式	按高置信度和类别平衡写入/加权	按时间滑窗写入当前预测 mask feature
是否可微参数	bank 本身是存储；attention/fusion 可训练	bank 本身是缓存；cross-attention/fusion 可训练

5.9 Figure 5 伪代码：从记忆读取到 mask 输出

Input: current feature r_t, current text F_txt, long-term banks, short-term MaskMemory

```
# Long-Term Memory Recall
c_txt = SelfAttn(Q=F_txt, K=TextMemory, V=TextMemory)
c_pair = SelfAttn(Q=r_t, K=WeightedPairMemory, V=VisualMemory_or_PairMemory)
r_LTM = CrossAttn(Q=c_txt, K=c_pair, V=c_pair)

# Short-Term Memory Recall
r_STM = CrossAttn(Q=r_LTM, K=MaskMemory, V=MaskMemory)

# Classifier
logits = Classifier(r_STM)
mask = sigmoid(logits)

# Update STM for next frame
f_mask_t = MaskFeatureExtractor(r_STM, mask)
MaskMemory.push(f_mask_t); MaskMemory.keep_last(K)
```

训练与推理差别：训练时 classifier 有 ground-truth mask 监督；推理时没有 GT，只能用当前预测 mask 生成 f_mask_t 并写入 STM。因此如果早期帧预测错，错误可能通过 STM 传播。论文主文没有详细讨论这个误差累积问题。

6. 损失函数设计：论文明确给出的部分与未明确部分

6.1 论文主文明确给出的损失

名称	公式/状态	作用模块	说明
Temporal contrastive alignment loss	L_{align}	CFTVA	让文本特征与正确时间/正确视觉目标特征更接近，与其他时间/不匹配特征更远。
Classifier loss	未给公式	Classifier	图中标注 loss，但主文未说明具体是 BCE/CE/Dice 等。
Memory weight uses sample loss	$w_i^{init}=1/(L_i+\epsilon)$	LTM	用于给 memory entry 加权，不是单独训练目标。

6.2 L_{align} 详细解释

$$L_{align} = - \log [\exp(\text{sim}(F_t^{\text{vis-align}}, F^{\text{txt}}) / \tau) / \sum_{t'} \exp(\text{sim}(F_{t'}^{\text{vis-align}}, F^{\text{txt}}) / \tau)]$$

- 目标：正确帧/正确目标与文本相似度最大。
- 分母：把其他帧或其他候选特征作为 negative。
- 效果：模型更容易区分“同样是银色车，但只有一辆在反方向移动”。
- 训练影响：主要训练 CFTVA 的投影层和 attention/fusion 参数，LLM 本体不训练。

6.3 分割损失的合理实现模板，但论文主文未明确

如果复现，需要自己在代码里确认。常见 query-driven 二值 mask 分割可以写成如下模板，但这不是论文主文给出的公式：

```
L_seg = BCEWithLogits(logits, y)
# or
L_seg = CE(logits, y)           # multi-class case
# optionally
L_seg_total = L_BCE + lambda_dice * L_Dice
```

一个合理的整体训练目标可写成：

```
L_total = L_seg + lambda_align * L_align + regularization_terms
# lambda_align and regularization terms are not specified in the main paper text.
```

严格结论：主文能确定：有 L_{align} ；有 classifier loss；memory entry 的权重依赖 L_i 。不能从主文确定：classifier loss 的具体形式、各 loss 权重、是否包含 Dice/Lovasz/auxiliary losses。

7. 训练状态和推理状态逐模块总结

模块	状态	说明
Point Branch / MLP / Point Transformer	训练	从点云学习点级几何特征。
Voxelization	不训练	离散化操作。
Voxel Encoder / sparse 3D Transformer	训练	学习体素级空间上下文。
Image Encoder	主文未明确	只确定输出 F_{img} 。冻结/训练需查代码。
LLaMA2-7B Text Encoder	冻结	论文明确 frozen pretrained。
Linear Projections W_{txt}/W_{vis}	训练	跨模态统一维度。
CFTVA Attention Blocks	训练	受 L_{align} 和分割损失监督。
STEM State-Space A/B	训练	学习历史状态和当前观测的融合比例。
STEM Transformer	训练	学习帧内空间上下文。
Transformer Encoders/Decoders	训练	分割特征编码和解码。
Text/Visual/Pair Memory Banks	动态更新	存储历史高置信度文本-视觉条目；不是普通反向传播参数。

模块	状态	说明
Mask Memory Bank	动态更新	推理时按时间写入最近 mask feature。
Memory Attention / Fusion	训练	读 memory 的网络参数可训练。
Classifier	训练	输出 mask logits。

7.1 训练阶段和推理阶段的差异

阶段	具体行为
训练阶段	有 ground-truth mask；计算 segmentation/classifier loss 和 L_align；memory 可根据样本 loss 写入/更新；参数通过反向传播更新。
推理阶段	没有 ground-truth mask；使用训练好的 encoder/CFTVA/STEM/memory attention/classifier；STM 用预测出的当前 mask feature 更新短期 memory。
潜在问题	STM 如果训练时使用 GT mask feature、推理时使用 predicted mask feature，会有 exposure bias；论文主文未说明具体策略。

8. 按模块列出的“输入-输出-含义-模型-损失-训练状态”总表

模块	输入	输出	模型/机制	损失	状态
F2: Point Input	$P_t: N \times t \times C_p$	原始点云	数据	无	无
F2: Point Branch	P_t	$F_t^{point}: N \times t \times D_p$	MLP / lightweight Point Transformer	来自下游 L_{seg}	训练
F2: Voxelization	P_t	$X_t^{voxel}: V \times t \times C_v$	体素化	无	不训练
F2: Voxel Encoder	X_t^{voxel}	$F_t^{voxel}: V \times t \times D_v$	sparse 3D Transformer	来自下游 L_{seg}	训练
F2: Image Encoder	$I_t: H \times W \times 3$	$F_t^{img}: M \times t \times D_i$	主文未明 backbone	来自下游损失或预训练	主文未明确
F2: LLM Encoder	query q	$F^{txt}: L \times D_{llm} / D_{llm}$	frozen LLaMA2-7B	无反传到 LLM	冻结
F3: Visual Aggregation	$F_{point}, F_{voxel}, F_{img}$	$F_{point'}, F_{voxel'}$	2D-3D CrossAttn	$L_{align} + L_{seg}$	训练
F3: Linear Projection	F_{txt}, F_{vis}	$F_{txt'}, F_{vis'}: * \times D$	Linear W_{txt}/W_{vis}	$L_{align} + L_{seg}$	训练
F3: Cross-Frame Attention	$F_{txt'}, [F_t, F_{t-1}, F_{t-2}]$	$F_{point-align}, F_{voxel-align}$	cross-modal cross-frame attention	$L_{align} + L_{seg}$	训练
F4: State-Space Update	$h_{[t-1]}$, aligned features	$h_t(v): D_s$	learnable A/B recurrence	L_{seg}	训练
F4: Transformer Update	h_t or aligned features	$z_t(v): D_s$	sparse 3D Transformer	L_{seg}	训练
F4: Norm Fusion	h_t, z_t	$f_{tilde_t}(v): D_s$	Norm + residual fusion	L_{seg}	训练
F5: LTM Banks	historical features	memory entries	Text/Visual/Pair banks	entry weight uses L_i	动态维护
F5: LTM Attention	current query + banks	$r_{LTM}: D$	self-attn + cross-attn	L_{seg}	训练
F5: STM Bank	recent mask features	MaskMemory	sliding memory cache	无直接损失	动态维护
F5: STM Attention	$r_{LTM} + \text{MaskMemory}$	$r_{STM}: D$	cross-attn	L_{seg}	训练
F5: Classifier	r_{STM} / final feature	mask logits	segmentation head	classifier loss, 主文未给公式	训练

9. 论文主文未给出但复现必须确认的细节

- Image Encoder 具体 backbone：主文未明确。需要查代码/补充材料。
- 文本特征池化方式： F^{txt} 是 token-level 还是 sentence-level，主文未明确。
- F_t^{vis} 的拼接方式：point token 与 voxel token 数量不同，主文没有说明具体数据结构。
- point-align 到 voxel v 的映射方式：STEM 公式需要 $F_{point-align}(v)$ ，但 point/voxel 索引不同。需确认是否通过 voxel pooling。
- State-space update 是否考虑 ego-motion compensation：主文没有展开。
- Classifier loss 具体形式：主文未给 BCE/CE/Dice 等公式。
- Memory Bank 更新频率、容量 M、淘汰策略、类别标签来源：主文没有完整展开。
- STM 训练时使用预测 mask feature 还是 GT mask feature：主文未说明。
- 3D 任务中重复输入三帧的并行处理细节：论文实验部分提到做法，但网络级细节需查实现。

10. 最终压缩理解

- Figure 2 是总流程：多模态输入 -> 对齐 -> 时空建模 -> 记忆召回 -> mask。
- Figure 3 的本质是 query-aware alignment：让文本 query 找到跨帧视觉/点云中的目标。
- Figure 4 的本质是 spatio-temporal stabilization：让目标在连续帧中不跳、不漂，在当前帧内部不碎、不粘。
- Figure 5 的本质是 hierarchical memory：LTM 负责跨场景语义经验，STM 负责当前序列的短期 mask 连续性。
- 已明确训练：除 frozen LLaMA2 和非参数 memory/voxelization 外，主要网络模块都需要训练。
- 已明确损失：CFTVA 的 temporal contrastive L_{align} ；classifier loss 存在但主文未给公式；memory 权重使用样本 loss 计算。

参考来源：MORE-STEM 论文 Figure 2-5、Section 3.1 Cross-Frame Text-Visual Alignment、Section 3.2 Spatio-Temporal Consistency Model、Section 3.3 Long-Short Memory Recall。